

WEEK 3 LAB REPORT

Control Flow & Functions

Student Name: Nwogu Amarachi Jennifer

Id: 27452

Course: Introduction to Big Data Analytics

Week: 3

Lab Topic: Control Flow & Functions

1. Decisions and Loops

Exercise 1.1 — The Grading Ladder

The grading ladder was implemented using if, elif, and else conditions.

Screenshot 1 — Exercise 1.1

```
[1]: # Exercise 1.1 – The Grading Ladder

scores = [91, 76, 64, 52, 43]

for score in scores:
    if score >= 85:
        grade = "A"
    elif score >= 70:
        grade = "B"
    elif score >= 60:
        grade = "C"
    elif score >= 50:
        grade = "D"
    else:
        grade = "F"

    print(f"Score {score} -> Grade {grade}")

Score 91 -> Grade A
Score 76 -> Grade B
Score 64 -> Grade C
Score 52 -> Grade D
Score 43 -> Grade F
```

Exercise 1.2 — Combined Conditions

The validate() function was used to check both the student's score and attendance.

Screenshot 2 — Exercise 1.2

```
[2]: # Exercise 1.2 – Combined Conditions

def validate(score, attendance):
    if score >= 50 and attendance >= 75:
        print("Validated")
    elif score >= 50 and attendance < 75:
        print("Blocked: low attendance")
    else:
        print("Failed")

validate(76, 92)
validate(76, 68)
validate(43, 95)

Validated
Blocked: low attendance
Failed
```

Question 1.3

Answer:

Scores from 70 to 84 would be classified as D instead of B because the score ≥ 50 condition would be checked first.

Screenshot 3 — Question 1.3

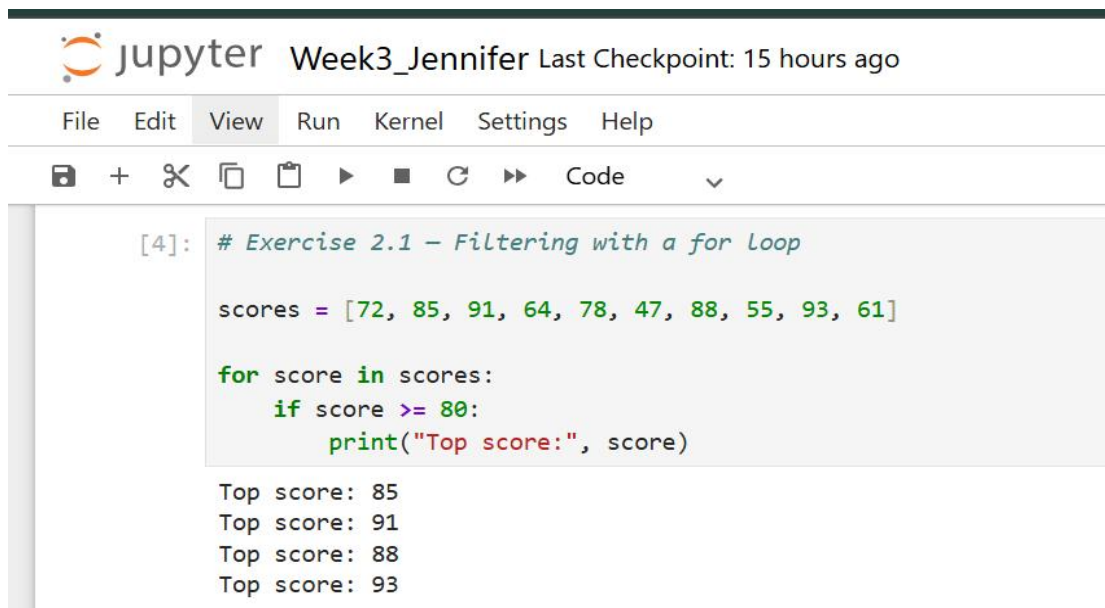
Q1.3 Answer:
Scores from 70 to 84 would be classified as D instead of B because the score ≥ 50 condition would be checked first.

2. Loops and Accumulators

Exercise 2.1 — Filtering with a for Loop

The program filtered the scores and displayed scores that were 80 or higher.

Screenshot 4 — Exercise 2.1



Jupyter Week3_Jennifer Last Checkpoint: 15 hours ago

File Edit View Run Kernel Settings Help

Code

```
[4]: # Exercise 2.1 – Filtering with a for loop

scores = [72, 85, 91, 64, 78, 47, 88, 55, 93, 61]

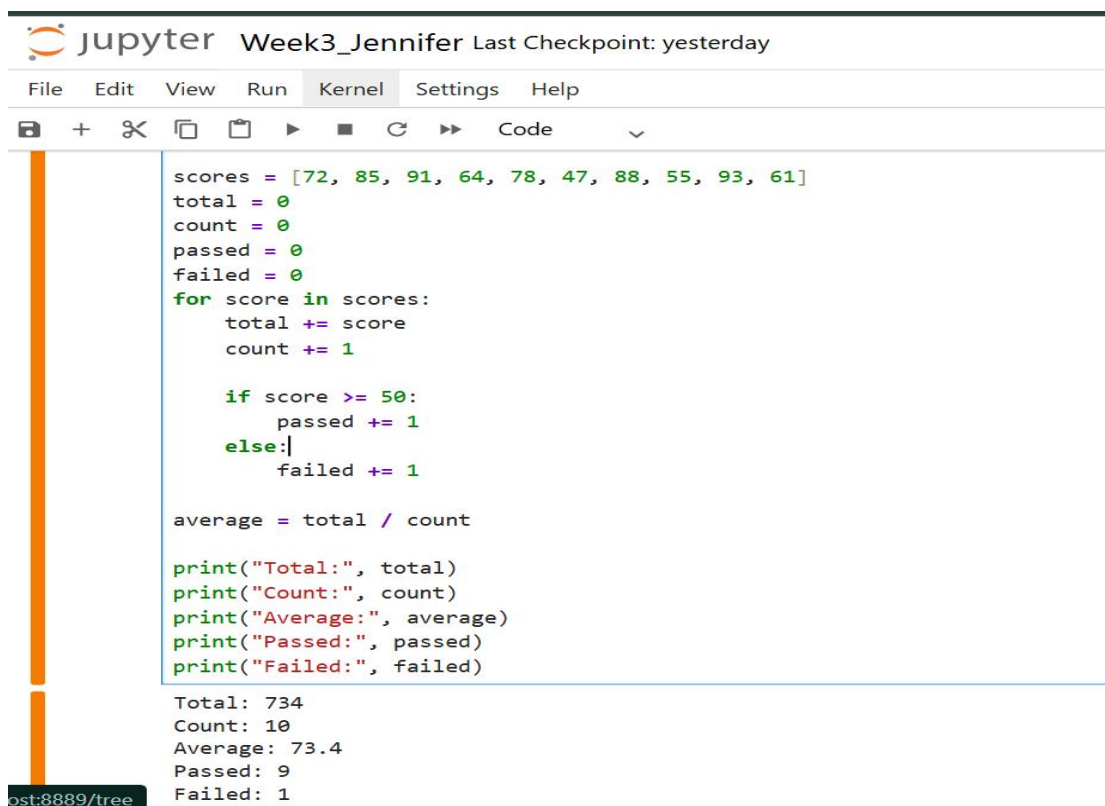
for score in scores:
    if score >= 80:
        print("Top score:", score)
```

Top score: 85
Top score: 91
Top score: 88
Top score: 93

Exercise 2.2 — Accumulator Pattern

The program calculated the total, number of scores, average, passed students, and failed students.

Screenshot 5 — Exercise 2.2



Jupyter Week3_Jennifer Last Checkpoint: yesterday

File Edit View Run Kernel Settings Help

Code

```
scores = [72, 85, 91, 64, 78, 47, 88, 55, 93, 61]
total = 0
count = 0
passed = 0
failed = 0
for score in scores:
    total += score
    count += 1

    if score >= 50:
        passed += 1
    else:
        failed += 1

average = total / count

print("Total:", total)
print("Count:", count)
print("Average:", average)
print("Passed:", passed)
print("Failed:", failed)
```

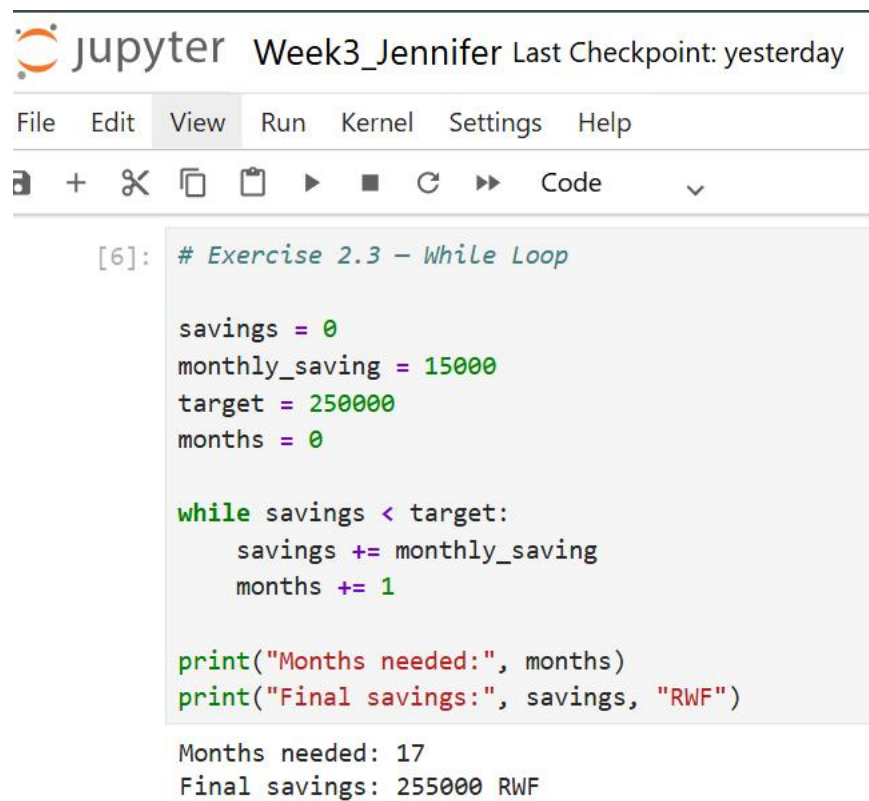
Total: 734
Count: 10
Average: 73.4
Passed: 9
Failed: 1

bst:8889/tree

Exercise 2.3 — While Loop

A while loop was used to calculate how many months were required to save enough money for a 250,000 RWF laptop.

Screenshot 6 — Exercise 2.3



The screenshot shows a Jupyter Notebook window titled "Week3_Jennifer" with a "Last Checkpoint: yesterday" status. The interface includes a menu bar with "File", "Edit", "View", "Run", "Kernel", "Settings", and "Help". Below the menu is a toolbar with icons for file operations and execution. The main area displays a code cell labeled "[6]:" containing the following Python code:

```
# Exercise 2.3 - While Loop

savings = 0
monthly_saving = 15000
target = 250000
months = 0

while savings < target:
    savings += monthly_saving
    months += 1

print("Months needed:", months)
print("Final savings:", savings, "RWF")
```

Below the code, the output of the cell is displayed:

```
Months needed: 17
Final savings: 255000 RWF
```

3. Functions

Exercise 3.1 — get_grade()

A function called `get_grade()` was created to return the appropriate letter grade.

Screenshot 7 — Exercise 3.1

jupyter Week3_Jennifer Last Checkpoint: yesterday

File Edit View Run Kernel Settings Help

Code

```
[7]: # Exercise 3.1 – get_grade function

def get_grade(score):
    """Return the letter grade for a score."""
    if score >= 85:
        return "A"
    elif score >= 70:
        return "B"
    elif score >= 60:
        return "C"
    elif score >= 50:
        return "D"
    else:
        return "F"

print(get_grade(91))
print(get_grade(76))
print(get_grade(43))
```

A
B
F

Exercise 3.2 — pass_rate()

A `pass_rate()` function was created to calculate the percentage of students who passed.

Screenshot 8 — Exercise 3.2

jupyter Week3_Jennifer Last Checkpoint: yesterday

File Edit View Run Kernel Settings Help

Code

```
[8]: # Exercise 3.2 – pass_rate function

def pass_rate(score_list):
    passed = 0

    for score in score_list:
        if score >= 50:
            passed += 1

    return (passed / len(score_list)) * 100

scores = [72, 85, 91, 64, 78, 47, 88, 55, 93, 61]

print(pass_rate(scores))
```

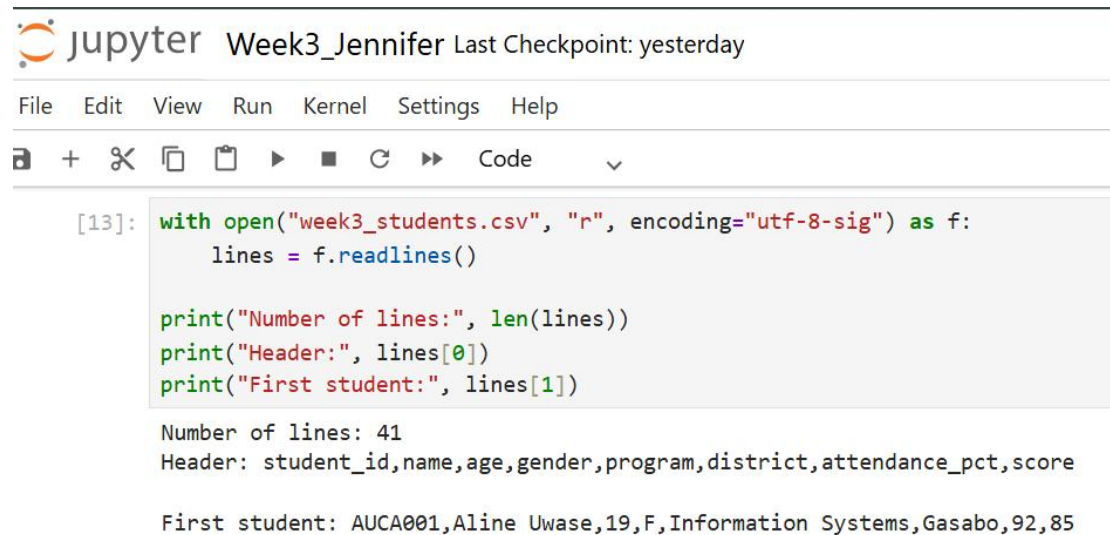
90.0

4. Files and CSV

Exercise 4.1 — Reading the CSV File

The week3_students.csv file was opened and read using Python. The file contained **41 lines**, consisting of 40 student records and one header row.

Screenshot 9 — Exercise 4.1



The screenshot shows a Jupyter Notebook window titled "Week3_Jennifer Last Checkpoint: yesterday". The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and code execution. The code cell [13] contains the following Python code:

```
[13]: with open("week3_students.csv", "r", encoding="utf-8-sig") as f:
      lines = f.readlines()

      print("Number of lines:", len(lines))
      print("Header:", lines[0])
      print("First student:", lines[1])
```

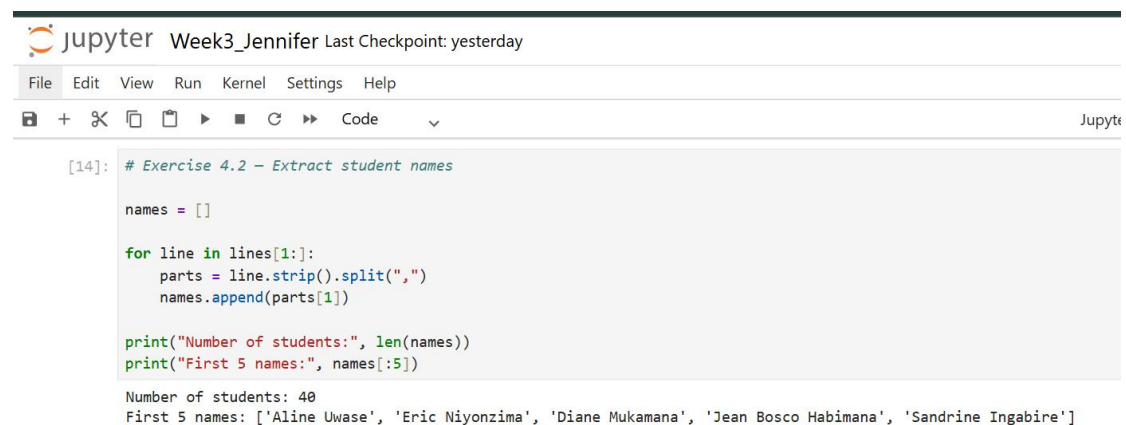
The output of the code is displayed below the code cell:

```
Number of lines: 41
Header: student_id,name,age,gender,program,district,attendance_pct,score
First student: AUCA001,Aline Uwase,19,F,Information Systems,Gasabo,92,85
```

Exercise 4.2 — Extracting Student Names

The student names were extracted from the CSV file using `split(",")`.

Screenshot 10 — Exercise 4.2



The screenshot shows a Jupyter Notebook window titled "Week3_Jennifer Last Checkpoint: yesterday". The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and code execution. The code cell [14] contains the following Python code:

```
[14]: # Exercise 4.2 - Extract student names

names = []

for line in lines[1:]:
    parts = line.strip().split(",")
    names.append(parts[1])

print("Number of students:", len(names))
print("First 5 names:", names[:5])
```

The output of the code is displayed below the code cell:

```
Number of students: 40
First 5 names: ['Aline Uwase', 'Eric Niyonzima', 'Diane Mukamana', 'Jean Bosco Habimana', 'Sandrine Ingabire']
```

5. Resilient Data Pipeline

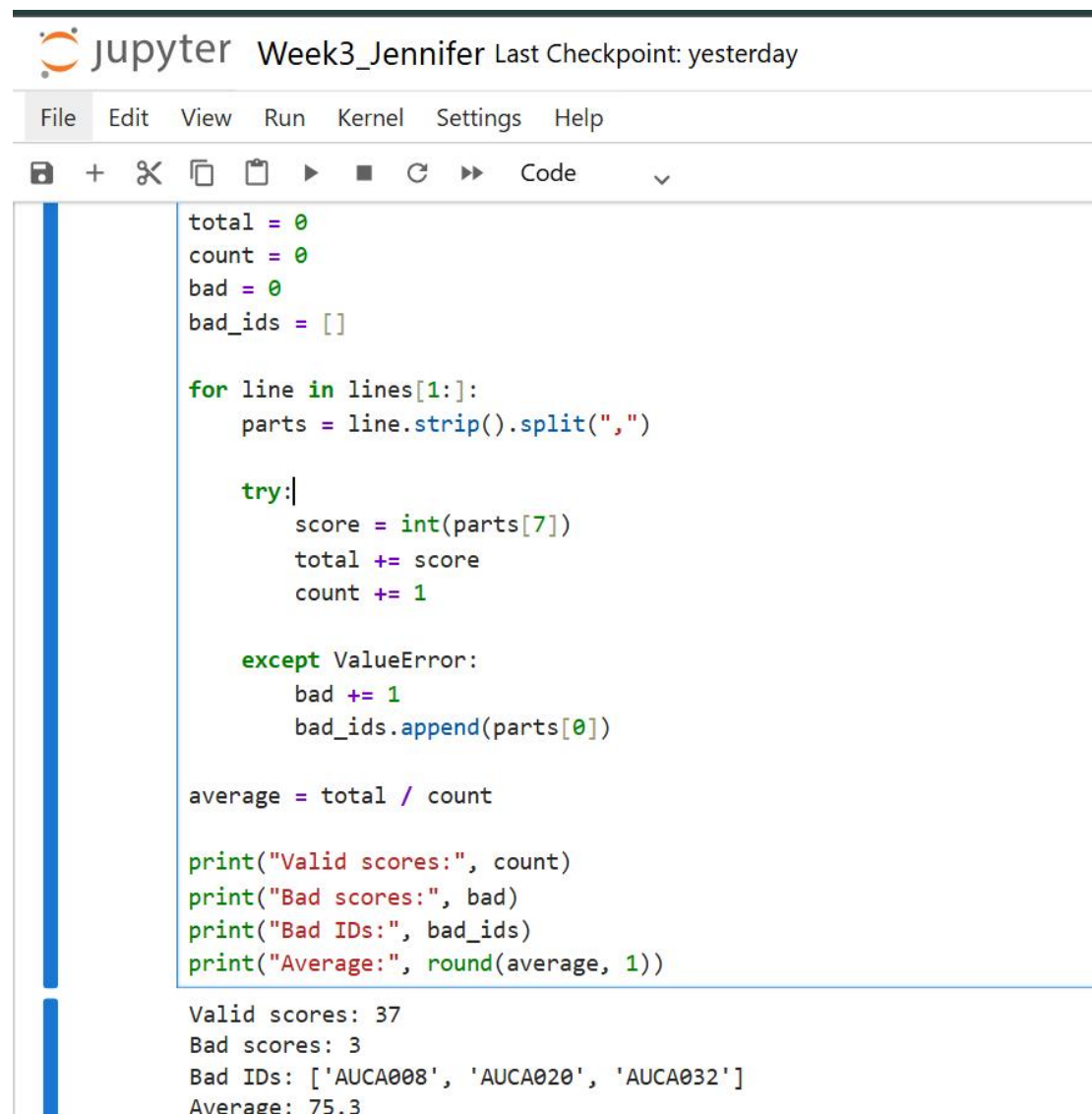
Exercise 5.1 — Handling Broken Scores

The program used try/except to handle invalid score values without stopping the program.

The three broken records were:

- **AUCA008** — N/A
- **AUCA020** — blank score
- **AUCA032** — absent

Screenshot 11 — Exercise 5.1



The screenshot shows a Jupyter Notebook window titled "Week3_Jennifer" with a "Last Checkpoint: yesterday" status. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and execution. The code editor contains the following Python code:

```
total = 0
count = 0
bad = 0
bad_ids = []

for line in lines[1:]:
    parts = line.strip().split(",")

    try:
        score = int(parts[7])
        total += score
        count += 1

    except ValueError:
        bad += 1
        bad_ids.append(parts[0])

average = total / count

print("Valid scores:", count)
print("Bad scores:", bad)
print("Bad IDs:", bad_ids)
print("Average:", round(average, 1))
```

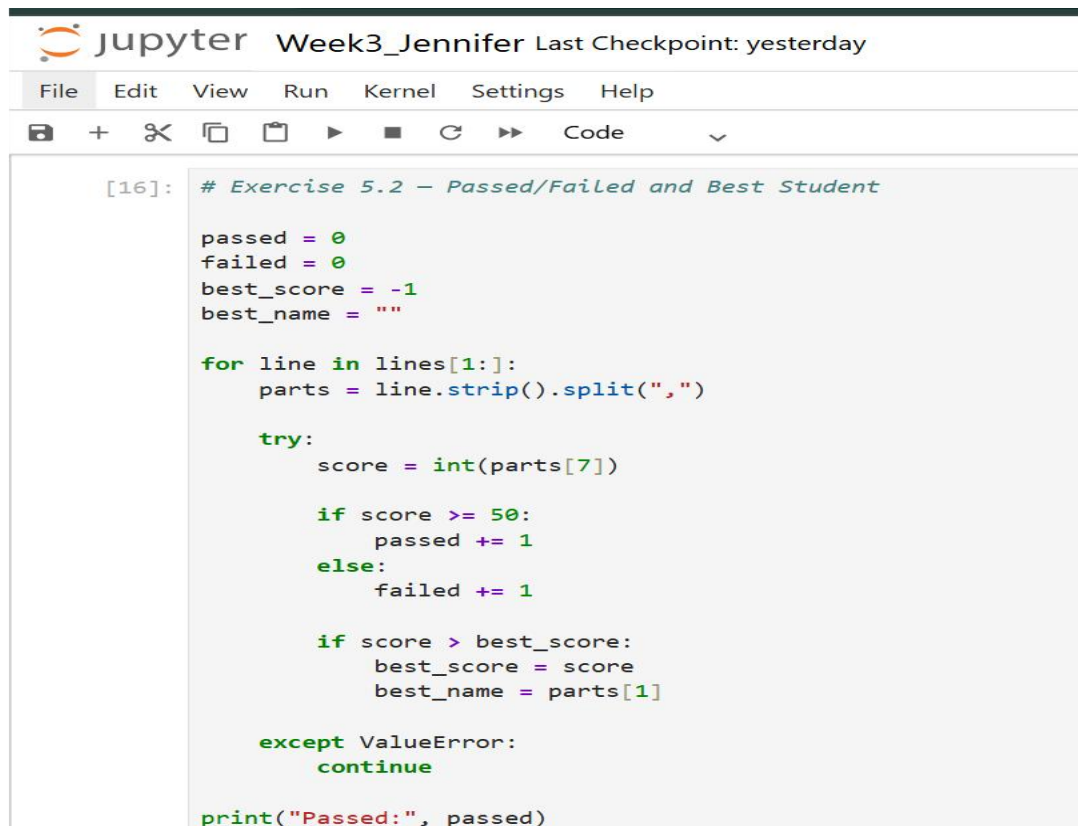
The output of the code is displayed below the editor:

```
Valid scores: 37
Bad scores: 3
Bad IDs: ['AUCA008', 'AUCA020', 'AUCA032']
Average: 75.3
```

Exercise 5.2 — Passed, Failed and Best Student

The program counted the passed and failed students and identified the student with the highest score.

Screenshot 12 — Exercise 5.2



```
[16]: # Exercise 5.2 – Passed/Failed and Best Student

passed = 0
failed = 0
best_score = -1
best_name = ""

for line in lines[1:]:
    parts = line.strip().split(",")

    try:
        score = int(parts[7])

        if score >= 50:
            passed += 1
        else:
            failed += 1

        if score > best_score:
            best_score = score
            best_name = parts[1]

    except ValueError:
        continue

print("Passed:", passed)
```

```
except ValueError:
    continue

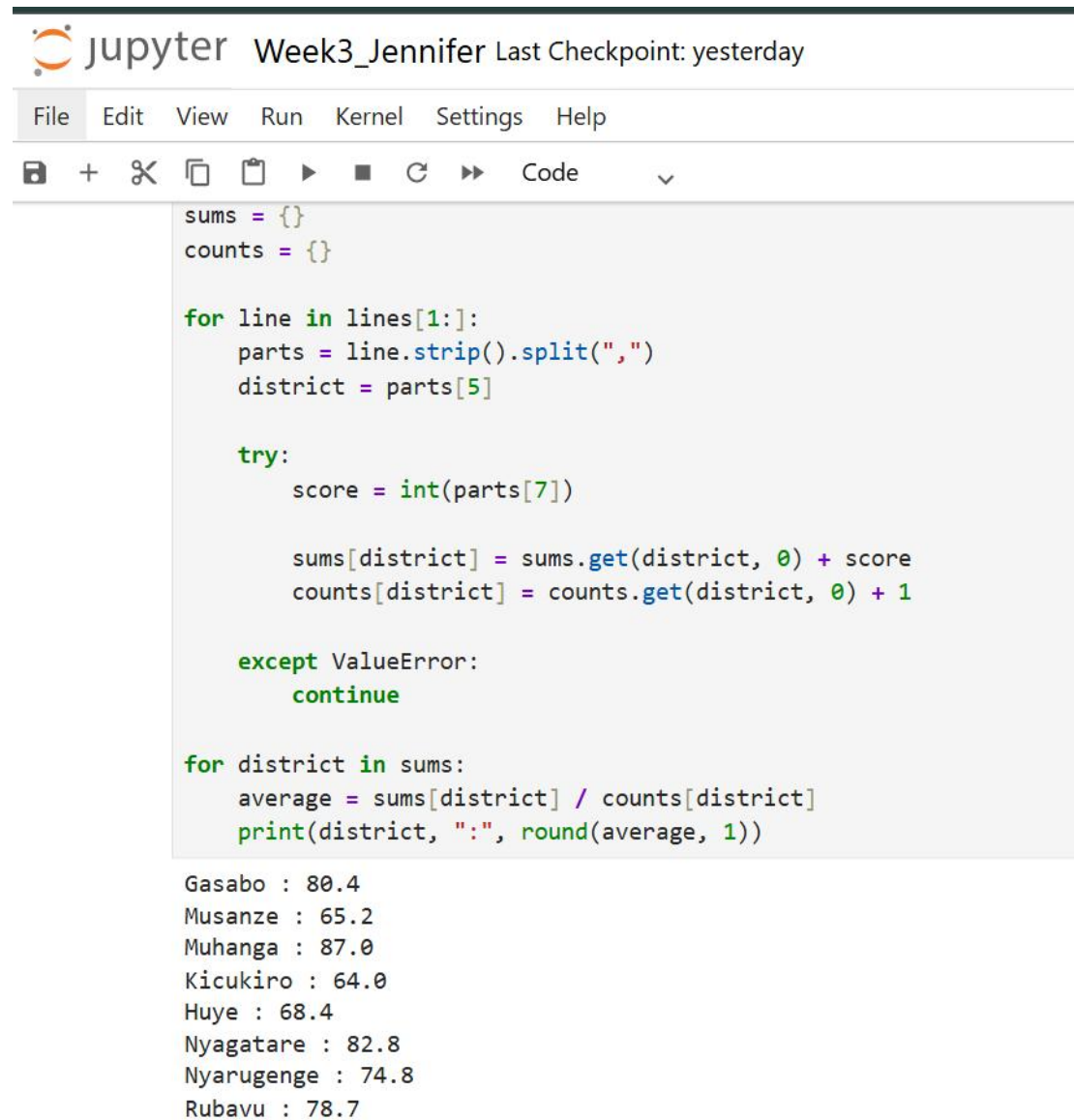
print("Passed:", passed)
print("Failed:", failed)
print("Top score:", best_score)
print("Top student:", best_name)
print("Grade:", get_grade(best_score))
```

```
Passed: 35
Failed: 2
Top score: 95
Top student: Bonaventure Nkurunziza
Grade: A
```

Exercise 5.3 — District Averages

The program calculated the average score for each district.

Screenshot 13 — Exercise 5.3



The screenshot shows a Jupyter Notebook interface. The top bar displays the Jupyter logo, the notebook name 'Week3_Jennifer', and the last checkpoint 'yesterday'. Below the top bar is a menu bar with 'File', 'Edit', 'View', 'Run', 'Kernel', 'Settings', and 'Help'. A toolbar with various icons is located below the menu bar. The main area contains a code cell with the following Python code:

```
sums = {}
counts = {}

for line in lines[1:]:
    parts = line.strip().split(",")
    district = parts[5]

    try:
        score = int(parts[7])

        sums[district] = sums.get(district, 0) + score
        counts[district] = counts.get(district, 0) + 1

    except ValueError:
        continue

for district in sums:
    average = sums[district] / counts[district]
    print(district, ":", round(average, 1))
```

The output of the code is displayed below the code cell:

```
Gasabo : 80.4
Musanze : 65.2
Muhanga : 87.0
Kicukiro : 64.0
Huye : 68.4
Nyagatare : 82.8
Nyarugenge : 74.8
Rubavu : 78.7
```

6. Bonus — Output Report

The program saved the results into a text file named `week3_report.txt`.

Screenshot 14 — Bonus Output

```
[18]: # Bonus – Save report to a text file

with open("week3_report.txt", "w") as f:
    f.write("WEEK 3 STUDENT REPORT\n")
    f.write("=====\n")
    f.write(f"Valid scores: {count}\n")
    f.write(f"Bad scores: {bad}\n")
    f.write(f"Average score: {round(average, 1)}\n")
    f.write(f"Bad IDs: {bad_ids}\n")
    f.write(f"Passed: {passed}\n")
    f.write(f"Failed: {failed}\n")
    f.write(f"Top student: {best_name}\n")
    f.write(f"Top score: {best_score}\n")
    f.write(f"Top grade: {get_grade(best_score)}\n")

print("Report saved successfully.")
```

Report saved successfully.

7. Week 3 Student Report

The generated report contained:

WEEK 3 STUDENT REPORT

```
=====
Valid scores: 37
Bad scores: 3
Average score: 78.7
Bad IDs: ['AUCA008', 'AUCA020', 'AUCA032']
Passed: 35
Failed: 2
Top student: Bonaventure Nkurunziza
Top score: 95
Top grade: A
```

Screenshot 15 — week3_report.txt

```
File Edit View H1 ▾ ⋮

WEEK 3 STUDENT REPORT
=====
Valid scores: 37
Bad scores: 3
Average score: 78.7
Bad IDs: ['AUCA008', 'AUCA020', 'AUCA032']
Passed: 35
Failed: 2
Top student: Bonaventure Nkurunziza
Top score: 95
Top grade: A
|
```

8. Reflection

The three broken records were **AUCA008**, **AUCA020**, and **AUCA032**. Their score values were N/A, blank, and absent. The program used try/except to handle these invalid scores without stopping the program. The invalid records were added to `bad_ids`, while the valid scores were used to calculate the average and other results.

9. Conclusion

The Week 3 lab demonstrated the use of **conditional statements, loops, accumulators, functions, file handling, exception handling, and CSV data processing in Python**. The program successfully processed the student dataset and handled the three invalid score records without stopping execution.
